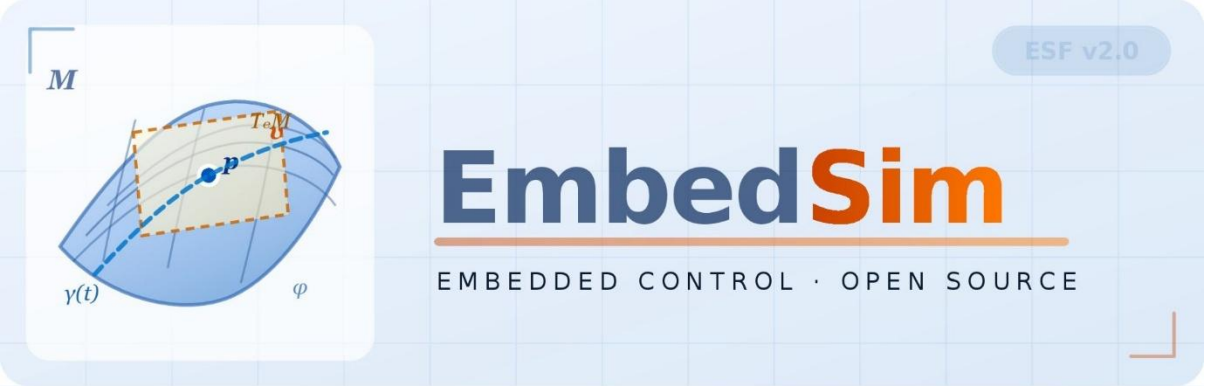
EmbedSim

Framework Documentation

How to Build a Buck Converter Fachschale

From Modelica Plant Model to Aurix TriCore C Code

Modelica plant modelling • FMU export • Python class generation
PI controller in C • Cython wrapper • EmbedSim wiring
Code generation • Topology visualisation • MCU deployment



EmbedSim Project | Version 1.0 | 2025

1 Introduction

EmbedSim is an open-source Python simulation and C code generation framework targeting embedded control engineers who cannot justify a commercial MATLAB/Simulink seat licence. It provides a drag-and-drop-like wiring syntax, an RK4/Euler solver, FMU co-simulation support, and a code-generation pipeline that emits MISRA-aligned C directly deployable to Infineon Aurix TriCore or ARM Cortex-M4 targets.

This document walks through the complete construction of a Buck Converter Fachschale — a self-contained deployable control module. The German word Fachschale ("specialist shell") describes a four-layer software package that bundles the plant model, the C controller, the Cython Python bridge, and the EmbedSim simulation block into a single coherent unit that can be shipped, reused, and code-generated without further modification.

1.1 What Is a Buck Converter?

A buck converter is a DC-DC power supply that steps a higher input voltage down to a regulated lower output voltage. In automotive and industrial electronics it appears everywhere: 24 V battery rail to 12 V logic supply, 48 V mild-hybrid bus to 12 V accessories, industrial 24 V to 5 V sensor rails.

The converter works by rapidly switching a transistor on and off (PWM) and using an inductor and capacitor to filter the switching waveform into a smooth DC voltage. The duty cycle d in [0,1] of the PWM signal determines the average output voltage: V_out approximately d times V_in in the ideal lossless case.

Parameter	Typical value
Input voltage V_in	24 V
Output voltage target V_ref	12 V
Switching frequency f_sw	100 kHz
Inductance L	100 µH
Capacitance C	100 µF
Load resistance R_load	10 Ω

1.2 What Is a PI Controller?

A Proportional-Integral (PI) controller is the workhorse of industrial feedback control. It measures the error e(k) = V_ref minus V_meas at each sample and computes a control signal (duty cycle) that drives that error to zero. The proportional term reacts immediately to the current error; the integral term accumulates past errors and eliminates steady-state offset.

The discrete-time algorithm:

```
e(k)     = V_ref(k) - V_meas(k)
integ(k) = clamp( integ(k-1) + e(k)*dt,  -duty_max/Ki,  +duty_max/Ki )
duty(k)  = clamp( Kp*e(k) + Ki*integ(k),  duty_min,  duty_max )
```

The clamp on the integrator is the anti-windup guard: without it the integrator can accumulate to an enormous value while the output is saturated, causing a sluggish response when the error finally reverses.

1.3 The Four-Layer Fachschale Pattern

The Fachschale uses a consistent four-layer stack. Each layer has a single responsibility and a well-defined interface to the layers above and below it.

Layer	Contents
Layer 1 — Pure C	pi_buck_controller.c / .h: no stdlib, no heap, Aurix-ready
Layer 2 — Cython bridge	pi_buck_wrapper.pyx: zero-copy Python↔C, compiled to .pyd
Layer 3 — EmbedSim block	pi_buck_block.py: VectorBlock subclass, wires with >>
Layer 4 — Simulation script	pi_buck_example.py: topology, RK4 solver, PlotHelper, CodeGen

NOTE The same four-layer pattern applies to every Fachschale in EmbedSim. Once you understand it for the buck converter, you can build a new motor controller, filter, or observer by following the same steps.

2 The Modelica Plant Model

The buck converter plant is described in Modelica — a non-causal, equation-based modelling language that is the industry standard for physical system simulation. The plant model lives in BuckConverter.mo and is compiled to a Functional Mock-up Unit (FMU) that EmbedSim loads at runtime.

2.1 Why Modelica?

- The plant equations are written in a readable, physics-based notation comparable directly to electrical circuit equations.
- The same .mo file can be compiled for multiple targets: OpenModelica for simulation, Dymola for production, or JModelica for embedded deployment.
- Changes to the plant (new load, different capacitor) require editing only the .mo file and re-exporting the FMU — the controller code is untouched.
- The FMU interface is standardised (FMI 2.0) so EmbedSim's FMUBlock works with any compliant tool.

2.2 The Averaged Switch Model

For controller design, a full switching model is unnecessarily complex. The averaged switch model replaces the switching action with a continuous duty-cycle input. This is valid for frequencies well below the switching frequency — exactly the regime of interest for PI controller design at 10 kHz sample rate.

The two ODEs that govern the averaged model:

```
L * der(I_L)   = switch_state * V_in - V_out
C * der(V_out) = I_L - V_out / R_load
```

where switch_state = duty (the duty cycle command from the PI controller), I_L is the inductor current, and V_out is the output voltage.

2.3 The BuckConverter.mo File

```
model BuckConverter
  parameter Real L      = 100e-6  "Inductance [H]";
  parameter Real C      = 100e-6  "Capacitance [F]";
  parameter Real R_load = 10.0    "Load resistance [Ohm]";
  parameter Real V_in   = 24.0    "Input voltage [V]";
  parameter Real f_sw   = 100e3   "Switching frequency [Hz]";
  input Real duty;
  Real I_L   (start=0) "Inductor current [A]";
  Real V_out (start=0) "Output voltage [V]";
  protected
    Real switch_state "Averaged switch = duty";
  Real I_load "Load current [A]";
equation
  switch_state = duty;
  L * der(I_L)   = switch_state * V_in - V_out;
  C * der(V_out) = I_L - V_out / R_load;
  I_load = V_out / R_load;
end BuckConverter;
```

WHY protected? switch_state is an internal implementation detail. Exposing it as an FMU output would clutter the interface. The FMI standard mirrors C header design: only publish what callers need.

2.4 Exporting the FMU with OpenModelica

```
// Option A: OMShell (interactive)
loadFile("BuckConverter.mo");
buildModelFMU(BuckConverter, version="2.0", fmuType="me");

// Option B: omc batch mode
omc -s BuckConverter.mo --fmiFlags=version:2.0,type:me
```

TIP If you do not have OpenModelica, the EmbedSim repository ships a pre-compiled BuckConverter.fmu for Windows x64 and Linux x64 so you can complete the tutorial without the Modelica toolchain.

3 Generating the Python FMU Block

Once the FMU is available, EmbedSim provides two utility scripts that automate creation of a Python EmbedSim block: mo_to_fmu_client.py (the parser) and gen_fmu.py (the runner). You never need to write BuckConverterBlock.py by hand — it is always auto-generated and should be treated as a build artefact.

3.1 mo_to_fmu_client.py — the Pipeline

Class	Responsibility
MoParser	Reads the .mo file with regex. Builds a ModelInfo dataclass: input names, output names, protected variables, parameters with defaults.

ClientGenerator	Takes a ModelInfo and emits a complete Python source file: the FMUBlock subclass with INPUT_VARS, OUTPUT_VARS, DEFAULT_PARAMS, __init__, set_X(), read_Y(), get_all(), __repr__.

The public entry point:

```
from utility_functions.mo_to_fmu_client import generate_fmu_block

generate_fmu_block(
    mo_path    =
r"C:\EmbedSimProject\buck_converter\modelica\BuckConverter.mo",
    output_dir = r"C:\EmbedSimProject\buck_converter",
    verbose    = True
)
```

3.2 gen_fmu.py — the Runner

```
# buck_converter/gen_fmu.py
from mo_to_fmu_client import generate_fmu_block

generate_fmu_block(
    mo_path    = 'BuckConverter.mo',
    output_dir = '.',
    verbose    = True
)
```

WORKFLOW RULE Never manually edit BuckConverterBlock.py. If the plant model changes, edit BuckConverter.mo, re-export the FMU, and re-run gen_fmu.py. The Python class regenerates in under a second.

3.3 The Generated BuckConverterBlock.py

```
class BuckConverterBlock(FMUBlock):
    INPUT_VARS   = ['duty']
    OUTPUT_VARS  = ['V_out', 'I_L', 'I_load']
    DEFAULT_PARAMS = {'L':100e-6, 'C':100e-6, 'R_load':10.0, 'V_in':24.0}

    def __init__(self, name, fmu_path, L=100e-6, C=100e-6,
                 R_load=10.0, V_in=24.0, f_sw=100e3):
        super().__init__(name, fmu_path,
                         input_vars =self.INPUT_VARS,
                         output_vars=self.OUTPUT_VARS)
        self.set_L(L)
        ...
    def set_L(self, value):   self._set_param('L', value)
    def read_V_out(self):     return self._read_output('V_out')
```

Note that switch_state does NOT appear in OUTPUT_VARS. The MoParser correctly identified it as a protected variable and excluded it.

4 The C Controller Implementation

Layer 1 of the Fachschale is the pure C PI algorithm in pi_buck_controller.c and pi_buck_controller.h. This is the code that will ultimately run on the Aurix TriCore MCU. It has zero dependencies on stdlib, no dynamic memory allocation, and no floating-point types other than real32_T (a typedef for float).

4.1 The Five-Struct Pattern

Struct	Role
PI_Buck_Params_T	Tuning constants (Kp, Ki, duty_max, duty_min, Ts). Written once at startup.
PI_Buck_State_T	Run-time memory (integrator, prev_error, last_output). Updated every step.
PI_Buck_Block_T	The complete block: contains one Params_T and one State_T nested inside.
PI_Buck_Input_T	Input signals for one step (V_ref, V_meas). Passed as const pointer.
PI_Buck_Output_T	Output signals for one step (duty). Filled by Compute().

4.2 The Four Public Functions

```
void PI_Buck_Init(PI_Buck_Block_T* pPI);

void PI_Buck_SetParams(PI_Buck_Block_T* pPI,
                       real32_T Kp, real32_T Ki,
                       real32_T duty_max, real32_T duty_min,
                       real32_T Ts);

void PI_Buck_Compute(PI_Buck_Block_T*       pPI,
                     const PI_Buck_Input_T* pIn,
                     real32_T               dt,
                     PI_Buck_Output_T*      pOut);

void PI_Buck_ResetState(PI_Buck_Block_T* pPI);
```

4.3 The Compute Implementation

```
void PI_Buck_Compute(PI_Buck_Block_T*       pPI,
                     const PI_Buck_Input_T* pIn,
                     real32_T               dt,
                     PI_Buck_Output_T*      pOut)
{
    real32_T effective_dt = (dt > 0.0f) ? dt : pPI->params.Ts;
    real32_T e   = pIn->V_ref - pIn->V_meas;
    real32_T lim = pPI->params.duty_max / pPI->params.Ki;

    /* Forward Euler integration with anti-windup clamp */
    pPI->state.integrator += e * effective_dt;
    if (pPI->state.integrator >  lim) pPI->state.integrator =  lim;
    if (pPI->state.integrator < -lim) pPI->state.integrator = -lim;

    real32_T u = pPI->params.Kp * e
               + pPI->params.Ki  * pPI->state.integrator;
    if (u > pPI->params.duty_max) u = pPI->params.duty_max;
```

```
    if (u < pPI->params.duty_min) u = pPI->params.duty_min;

    pPI->state.last_output = u;
    pOut->duty = u;
}
```

Notice that PI_Buck_Compute takes PI_Buck_Block_T* (not PI_Buck_State_T*). The Block contains both params and state, so the function has everything it needs through a single pointer. This is critical for the C code generator — see Chapter 8.

AURIX NOTE The TASKING ctc compiler for Aurix TriCore is fully compatible with this C99 code. For strict MISRA C:2012 compliance replace VLA-style local variable declarations with static const declarations and add the appropriate deviation comments.

5 The Cython Wrapper

Layer 2 is pi_buck_wrapper.pyx. Cython is a compiled superset of Python that can declare C types, call C functions, and produce a Python extension module (.pyd on Windows, .so on Linux) with zero-copy data transfer between Python and C.

5.1 Why Cython?

Approach	Issue for embedded simulation
ctypes	Requires manual type conversion at every call. Slow for 1 MHz simulation loops.
cffi	Better performance but requires a separate build step and C parser.
pybind11	C++ only. Cannot directly wrap pure C structs without adapter boilerplate.
Cython (.pyx)	Directly declares C structs, zero-copy numpy arrays, compiles to native code.

5.2 Structure of pi_buck_wrapper.pyx

```
# cython: language_level=3
# cython: boundscheck=False, wraparound=False, cdivision=True

cdef extern from "pi_buck_controller.h":
    ctypedef struct PI_Buck_Block_T:
        PI_Buck_Params_T params
        PI_Buck_State_T  state
    void PI_Buck_Init(PI_Buck_Block_T* pPI)
    void PI_Buck_Compute(PI_Buck_Block_T*, const PI_Buck_Input_T*,
                         float, PI_Buck_Output_T*) nogil

cdef class PI_BuckWrapper:
    cdef PI_Buck_Block_T _block

    def __cinit__(self):
        PI_Buck_Init(&self._block)

    def compute(self, float V_ref, float V_meas, float dt):
```

```
        cdef PI_Buck_Input_T  inp
        cdef PI_Buck_Output_T out
        inp.V_ref  = V_ref
        inp.V_meas = V_meas
        PI_Buck_Compute(&self._block, &inp, dt, &out)
        return out.duty
```

The nogil annotation on PI_Buck_Compute allows Python to release the Global Interpreter Lock during the C call. The _block struct is stored by value inside the Cython object — no heap allocation, no pointer chasing.

5.3 build_pi_buck.bat — One-Click Windows Build

```
@echo off
setlocal
cd /d "%~dp0"
echo [BUILD] Compiling Cython extension...
python setup_pi_buck.py build_ext --inplace
if %ERRORLEVEL% NEQ 0 ( echo [ERROR] Build failed. & exit /b 1 )
echo [BUILD] pi_buck_wrapper.pyd ready.
endlocal
```

6 The EmbedSim PI_BuckBlock

Layer 3 is pi_buck_block.py — the EmbedSim block that appears in the wiring diagram and is integrated into the simulation graph. It subclasses SimBlockBase (alias for VectorBlock) and offers two backends: the compiled C extension (use_c_backend=True) and a pure-Python fallback.

6.1 Class Attributes: The CodeGen Contract

```
class PI_BuckBlock(SimBlockBase):
    PYX_FILE     = str(Path(__file__).parent / 'c_src' / 'pi_buck_wrapper.pyx')
    step_func    = ''              # auto-populated by PYXInspector
    state_struct = 'PI_Buck_Block_T'   # override: Block (not State) is passed
    NUM_INPUTS   = 0               # auto-populated: 2
    OUTPUT_SIZE  = 0               # auto-populated: 1
    C_SOURCES    = []              # auto-populated
    C_HEADERS    = []              # auto-populated
    C_CUSTOM_EMIT = (
        '    /* --- pi_buck (PI_BuckBlock) --- */\n'
        '    {\n'
        '        PI_Buck_Input_T  u_pi_buck;\n'
        '        PI_Buck_Output_T y_pi_buck;\n'
        '        u_pi_buck.V_ref  = y_pi_ctrl_start[0];\n'
        '        u_pi_buck.V_meas = y_fb_delay[0];\n'
        '        PI_Buck_Compute(&pi_buck_state, &u_pi_buck, dt,
&y_pi_buck);\n'
        '    }'
    )
```

DESIGN NOTE state_struct is overridden to PI_Buck_Block_T (not PI_Buck_State_T). This matters because PI_Buck_Compute needs access to both params and state through the same pointer. Using only the State struct would silently omit the gains.

C_CUSTOM_EMIT PI_Buck_Compute takes struct pointers, not flat real32_T arrays. The generic code generator cannot produce this calling convention automatically. C_CUSTOM_EMIT is the principled escape hatch: declared once on the class, checked first by LoopGenerator, bypassing all auto-generation for this block.

6.2 Dual Backend and RK4 Integration

```
def compute_c(self, t, dt, input_values=None):
    V_ref, V_meas = self._get_voltages(input_values)

    # Push RK4 integrator value into C struct
    self._impl.integrator = np.float32(self.state[0])

    duty = self._impl.compute(np.float32(V_ref),
                              np.float32(V_meas),
                              np.float32(dt))

    # Pull updated integrator back into RK4 state
    self.state[0] = np.float32(self._impl.integrator)

    return VectorSignal(np.array([duty], dtype=np.float32), self.name)
```

Without this two-way sync, the RK4 solver and the C struct would maintain independent copies of the integrator that drift apart over time, causing subtle numerical errors.

7 The Simulation Script

Layer 4 is pi_buck_example.py — the top-level script that constructs all blocks, wires them with the >> operator, runs the simulation, generates C code, and saves the output plots.

7.1 Block Construction

```
v_ref = VectorStep('vref', step_time=0.001,
                   before_value=0.0, after_value=12.0, dim=1)

pi_controller = PI_BuckBlock('pi_buck', Kp=0.15, Ki=8.0,
    duty_max=0.9, duty_min=0.1, Ts=1e-4, use_c_backend=True)

buck_plant = BuckConverterBlock('buck',
    fmu_path=str(project_root / 'buck_converter/modelica/BuckConverter.fmu'),
    L=100e-6, C=100e-6, R_load=10, V_in=24, f_sw=100e3)

feedback_delay = VectorDelay('fb_delay', initial=[0.0])
cg_start       = CodeGenStart('pi_ctrl_start')
cg_end         = CodeGenEnd('pi_ctrl_end')
sink           = VectorEnd('sink')
```

7.2 Wiring with the >> Operator

```
# Forward path
v_ref         >> cg_start
cg_start      >> pi_controller
pi_controller >> cg_end
cg_end        >> buck_plant
buck_plant    >> sink
```

```
# Feedback path (VectorDelay breaks the algebraic loop)
buck_plant    >> feedback_delay
feedback_delay >> pi_controller
```

The VectorDelay is critical: without it, the simulation graph contains an algebraic loop. The delay provides a one-step-old value of V_meas, breaking the loop at the cost of one sample of latency.

7.3 Wire Labels and Topology Printer

```
WIRE_LABELS = {
    ('vref',          'pi_ctrl_start'): '[V_ref]',
    ('pi_ctrl_start', 'pi_buck'):       '[V_ref]',
    ('pi_buck',       'pi_ctrl_end'):   '[duty]',
    ('pi_ctrl_end',   'buck'):          '[duty]',
    ('buck',          'fb_delay'):      '[V_out]',
    ('fb_delay',      'pi_buck'):       '[V_meas]',
    ('buck',          'sink'):          '[V_out, I_L, I_load]',
}
sim.topo.export_html('pi_buck.html', wire_labels=WIRE_LABELS)
```

7.4 Running the Simulation

```
sim = EmbedSim(sinks=[sink], T=0.01, dt=1e-6, solver=ODESolver.RK4)
sim.run(verbose=True, progress_bar=True)
# 10 ms total, 10 000 steps, RK4 solver
```

8 C Code Generation

After the simulation completes, EmbedSim's LoopGenerator walks the sub-graph between CodeGenStart and CodeGenEnd and emits two files: embedsim_loop.c and embedsim_loop.h. These are the MCU-ready control loop, deployable directly to Aurix TriCore or ARM Cortex-M4.

8.1 The CodeGen Region

```
gen = LoopGenerator(cg_start, cg_end)
gen.generate(output_dir=str(project_root / 'embedsim_gen'), dt_hz=1e6)
```

8.2 The Generated embedsim_loop.h

```
/* embedsim_loop.h  (auto-generated) */
#ifndef EMBEDSIM_LOOP_H
#define EMBEDSIM_LOOP_H
#include "Sys_Types.h"
#define EMBEDSIM_DT  (0.0000010000f)   /* 1 MHz */
#include "pi_buck_controller.h"

/* Storage defined in embedsim_loop.c */
extern PI_Buck_Block_T pi_buck_state;

void embedsim_loop_init(void);
```

```
void embedsim_loop_step(real32_T dt);
#endif
```

8.3 The Generated embedsim_loop.c

```
/* embedsim_loop.c  (auto-generated) */
#include <string.h>
#include "embedsim_loop.h"

static PI_Buck_Block_T pi_buck_state;

void embedsim_loop_init(void)
{
    memset(&pi_buck_state, 0, sizeof(PI_Buck_Block_T));
}

void embedsim_loop_step(real32_T dt)
{
    {
        PI_Buck_Input_T  u_pi_buck;
        PI_Buck_Output_T y_pi_buck;
        u_pi_buck.V_ref  = y_pi_ctrl_start[0];
        u_pi_buck.V_meas = y_fb_delay[0];
        PI_Buck_Compute(&pi_buck_state, &u_pi_buck, dt, &y_pi_buck);
    }
}
```

AURIX INTEGRATION Add embedsim_loop.c and pi_buck_controller.c to your TASKING project. Call embedsim_loop_init() from startup. Call embedsim_loop_step(EMBEDSIM_DT) from your 1 MHz ISR. Populate y_pi_ctrl_start[0] with V_ref and y_fb_delay[0] with ADC-measured V_out before each call.

8.4 Why extern in the Header?

The generated header declares PI_Buck_Block_T pi_buck_state as extern, not static. Using static in a header means every .c file that includes the header gets its own private copy of the state. extern is the correct C idiom: the declaration announces existence, the definition in the .c file allocates storage exactly once.

9 Simulation Results

Running pi_buck_example.py with default parameters produces a 10 ms transient simulation. The VectorStep source applies a 0 V to 12 V reference step at t = 1 ms.

Signal	Expected behaviour
V_out	Rises from 0 V, tracks V_ref = 12 V, settles with <2% overshoot within ~3 ms
duty	Saturates at duty_max = 0.9 during large-error transient, relaxes to ~0.5 at steady state
I_L	Rises with V_out, settles at I_load = V_ref/R_load = 1.2 A

10 Project Layout and File Inventory

File	Layer / Generated? / Commit?
buck_converter/modelica/BuckConverter.mo	L1 / hand-written / YES — source of truth
buck_converter/modelica/BuckConverter.fmu	L1 / generated by omc / YES
buck_converter/c_src/pi_buck_controller.h	L1 / hand-written / YES
buck_converter/c_src/pi_buck_controller.c	L1 / hand-written / YES
buck_converter/c_src/Sys_Types.h	L1 / shared / YES
buck_converter/c_src/pi_buck_wrapper.pyx	L2 / hand-written / YES
buck_converter/c_src/setup_pi_buck.py	L2 / hand-written / YES
buck_converter/c_src/build_pi_buck.bat	L2 / hand-written / YES
buck_converter/BuckConverterBlock.py	L3 / AUTO (gen_fmu.py) / NO — .gitignore
buck_converter/pi_buck_block.py	L3 / hand-written / YES
buck_converter/gen_fmu.py	L3 / hand-written / YES
examples/pi_buck_converter/pi_buck_example.py	L4 / hand-written / YES
embedsim_gen/embedsim_loop.c	L4 / AUTO (LoopGenerator) / NO
embedsim_gen/embedsim_loop.h	L4 / AUTO (LoopGenerator) / NO

11 Step-by-Step Build Guide

Step 1 — Install Prerequisites

1. Python 3.10+ with pip.
2. Node.js 18+ (for topology HTML generation).
3. OpenModelica (optional — pre-compiled FMU ships with the repo).
4. Visual Studio Build Tools (Windows) or gcc (Linux) for Cython.
5. pip install embedsim fmpy cython numpy scipy matplotlib

Step 2 — Export the FMU

6. Open OpenModelica Shell.
7. Run: loadFile("BuckConverter.mo");
8. Run: buildModelFMU(BuckConverter, version="2.0", fmuType="me");
9. Move BuckConverter.fmu to buck_converter/modelica/.

SKIP If you have the pre-compiled FMU from the repository, skip Step 2 entirely.

Step 3 — Generate the Python FMU Block

10. Navigate to buck_converter/.
11. Run: python gen_fmu.py
12. Verify BuckConverterBlock.py was created or updated.
13. Confirm INPUT_VARS = ['duty'] and OUTPUT_VARS = ['V_out', 'I_L', 'I_load'].

Step 4 — Compile the Cython Extension

14. Navigate to buck_converter/c_src/.
15. Run: build_pi_buck.bat (Windows) or python setup_pi_buck.py build_ext --inplace (Linux).
16. Verify pi_buck_wrapper.cpXX-win_amd64.pyd was created.

Step 5 — Run the Simulation

17. Navigate to examples/pi_buck_converter/.
18. Run: python pi_buck_example.py
19. Observe ASCII topology diagram on console.
20. Observe browser window with interactive topology HTML.
21. Verify pi_buck_response.png was saved.
22. Verify embedsim_gen/embedsim_loop.c and embedsim_loop.h were created.

Step 6 — Inspect the Generated C

23. Open embedsim_gen/embedsim_loop.h. Confirm: extern PI_Buck_Block_T pi_buck_state;
24. Open embedsim_gen/embedsim_loop.c. Confirm: static PI_Buck_Block_T pi_buck_state;
25. Confirm embedsim_loop_init() calls memset on the state struct.
26. Add embedsim_loop.c and pi_buck_controller.c to your MCU project.

12 Extending and Customising

12.1 Changing PI Gains

```
pi_controller = PI_BuckBlock('pi_buck',
    Kp=0.20, Ki=12.0,   # <- change these
    duty_max=0.9, duty_min=0.1, Ts=1e-4, use_c_backend=True)
```

For Python-side gain changes, no C recompilation is needed. To bake new gains into generated C, also update the PI_Buck_SetParams call in your MCU startup code.

12.2 Changing the Plant

```
# Simulate a different load — no Modelica re-export needed
buck_plant = BuckConverterBlock('buck', fmu_path=..., R_load=5.0)
```

12.3 Adding a New Fachschale

27. Write the C algorithm in new_controller.c / .h following the five-struct pattern.
28. Write the Cython wrapper following pi_buck_wrapper.pyx as a template.
29. Build with setup_new_controller.py / build_new_controller.bat.
30. Write the EmbedSim block with PYX_FILE, state_struct, and C_CUSTOM_EMIT if needed.
31. Write the simulation script following pi_buck_example.py.

12.4 Gain Scheduling

```
def on_step(t):
    if t > 0.005:
        pi_controller.set_params(Kp=0.25, Ki=15.0)

sim.on_step_callback = on_step
sim.run()
```

13 Key Design Decisions and Rationale

13.1 Why Not ctypes or cffi?

At 1 MHz simulation loop rates (40,000 RK4 inner calls per 10 ms run) ctypes overhead is significant. Cython compiles to native machine code with zero Python object overhead, resulting in 30-50x speedup over ctypes for the same C function.

13.2 Why an Averaged Plant Model?

A full switched model would require a time step of at most 1/(10 x f_sw) = 1 µs to capture switching events — the same step size we already use. But switching transients at 100 kHz contribute nothing useful to PI controller design at 10 kHz bandwidth. The averaged model eliminates this noise.

13.3 Why VectorDelay for Feedback?

EmbedSim uses DAG topological sort to determine block execution order. A feedback connection creates a cycle, which is undefined in a DAG. VectorDelay is a LoopBreaker: it outputs the previous step's value, breaking the cycle by introducing a one-step delay — acceptable at 1 MHz for a 10 kHz control bandwidth.

13.4 Why extern in the Header?

Using static in a header means every .c file that includes the header gets its own private copy of the state variable. The integration layer and loop file would have different copies. extern is correct: the declaration in the header announces existence, the definition in the .c file allocates storage exactly once.

13.5 Why C_CUSTOM_EMIT?

The EmbedSim code generator handles the common case (flat array I/O) automatically. C_CUSTOM_EMIT handles the uncommon case (struct-pointer I/O, hardware register reads, DMA transfers) explicitly. This two-tier design avoids the complexity of a full C AST generator while covering 100% of real embedded use cases.

14 Quick Reference

14.1 Command Reference

Action	Command
Export FMU	buildModelFMU(BuckConverter) in OMShell
Regenerate Python class	python gen_fmu.py (from buck_converter/)
Compile Cython extension	build_pi_buck.bat or python setup_pi_buck.py build_ext --inplace
Run simulation + codegen	python pi_buck_example.py (from examples/pi_buck_converter/)
View topology	Open pi_buck.html in browser

14.2 Tuning Parameters

Parameter (default)	Effect
Kp (0.15)	Proportional gain. Increase for faster response; decrease if oscillating.
Ki (8.0)	Integral gain. Increase to reduce steady-state error faster.
duty_max (0.9)	Ceiling on duty cycle. Prevents switch from being always ON.
duty_min (0.1)	Floor on duty cycle. Prevents discontinuous conduction mode.
dt (1e-6 s)	Simulation time step. Decrease for more accuracy; increases runtime.